RMBench: Memory-Dependent Robotic Manipulation Benchmark with Insights into Policy Design

Tianxing Chen*[1], Yuran Wang*[2,3], Mingleyang Li*[2], Yan Qin*[4], Hao Shi[5], Zixuan Li[6], Yifan Hu[2], Yingsheng Zhang[5], Kaixuan Wang[1], Yue Chen[2], Hongcheng Wang[2], Junjie Wang[5], Tianhang Yang[2], Renjing Xu[4], Ruihai Wu[2], Yao Mu[7], Yaodong Yang[2,3], Hao Dong†[2], Ping Luo†[1]

∗ Equal contribution, † Corresponding authors

Website: https://RMBench.github.io Code: https://github.com/robotwin-Platform/rmbench

arXiv:2603.01229v4 [cs.RO] 15 Oct 2026

***Abstract*— Robotic manipulation policies have made rapid progress in recent years, yet most existing approaches give limited consideration to memory capabilities. Consequently, they struggle to solve tasks that require reasoning over historical observations and maintaining task-relevant information over time, which are common in real-world manipulation. Although several memory-aware policies have been proposed, systematic evaluation of memory-dependent manipulation remains under-explored, and the relationship between architectural choices and memory performance remains unclear. To address this gap, we introduce RMBench, a simulation benchmark of nine manipulation tasks spanning two levels of memory complexity for systematic evaluation of policy memory capabilities. We further propose Mem-0, a modular manipulation policy whose explicit memory components can be ablated independently. Evaluating eight policies on the nine simulated tasks and four policies on three physical tasks, we identify memory-related limitations in existing policies and show that the benefit of a memory component depends on whether the same information is already available elsewhere in the policy hierarchy, with subtask termination detection rather than memory representation forming the dominant bottleneck. Website: rmbench-anonymous.github.io.**

I. Introduction

Recent progress in robotic manipulation has demonstrated strong capabilities across a wide range of tasks. Modern policies such as Pi0.6 [1] and RDT2 [2] achieve impressive performance in flexible object manipulation and fine-grained skills, including complex activities like coffee making. Nevertheless, most existing robotic policies are primarily designed for short-horizon tasks and fine-grained manipulation. These approaches typically rely on a fixed-length window of recent observations, implicitly assuming that the underlying decision process is approximately Markovian.

In contrast, memory-dependent tasks are ubiquitous in real-world robotic applications. Such tasks are inherently non-Markovian, as past observations and actions may influence future decisions over extended temporal horizons. Examples include remembering the location of previously placed objects and reasoning over multiple attempts after forgetting a password. These tasks are both challenging and practically important, as they require robots to retain, retrieve, and utilize information beyond short-term sensory inputs.

Motivated by this challenge, several recent works have begun to explore memory-aware robotic policies. Approaches such as MemoryVLA [3], MemER [4], CronusVLA [5], and SAM2Act [6] incorporate explicit memory mechanisms to address long-horizon and memory-dependent decision making. Despite these efforts, the field currently lacks a systematically designed experimental platform for evaluating and analyzing robotic policies under long-term memory requirements. In particular, there is limited understanding of the underlying mechanisms that make memory strategies effective for robotic manipulation.

Existing benchmarks only partially bridge this gap. MemoryBench [6] comprises seven single-arm manipulation tasks that involve memory, yet only three can be reliably reproduced in simulation, and the benchmark provides limited guidance on principled task design. MIKASA [7] introduces 32 memory-related manipulation tasks, but its formulation is largely tailored to reinforcement learning rather than general imitation learning. LIBERO-Long [8] offers ten long-horizon tasks, though these tasks do not explicitly demand memory since all task-relevant information remains observable throughout execution.

To address these limitations, we first introduce Task Memory Complexity, a principled metric for characterizing memory requirements in robotic manipulation tasks. This metric provides a systematic way to classify memory-dependent tasks and serves as a guideline for task design. Based on this formulation, we propose **RMBench**, a robotic manipulation benchmark built on the RoboTwin 2.0 [9] platform. RMBench consists of nine dual-arm manipulation tasks spanning different levels of task memory complexity, enabling large-scale and controlled studies of memory retention and utilization in robotic manipulation.

Furthermore, we propose **Mem-0**, a novel memory-oriented robotic policy designed with modular components that can be easily replaced or reconfigured. Mem-0 adopts a dual-system architecture in which a Subtask End Classifier detects the boundaries between task stages, allowing structured memory usage across long horizons. Through systematic ablation studies of Mem-0, we analyze which design components

[1]MMLab@HKU, [2]PKU, [3]PsiBot, [4]HKUST (GZ), [5]THU, [6]SZU, [7]SJTU. Correspondence to: Tianxing Chen <chentianxing@connect.hku.hk>, Ping Luo <pluo@hku.hk>, Hao Dong <hao.dong@pku.edu.cn>.

are critical for effective memory in robotic manipulation and derive insights for future policy design.

Our main contributions are summarized as follows:

- We introduce Task Memory Complexity, a principled metric for categorizing memory-dependent robotic manipulation tasks, and propose **RMBench**, a simulation benchmark comprising nine memory-centric tasks based on this metric.
- We propose **Mem-0**, a memory-oriented robotic manipulation policy featuring a dual-system architecture with a Subtask End Classifier for flexible memory usage.
- We conduct comprehensive evaluations of representative state-of-the-art policies on RMBench and perform detailed ablation studies of Mem-0, revealing which policy design mechanisms are most beneficial for memory in robotic manipulation.

II. Related Work

A. Robotic Manipulation Benchmarks

Numerous simulation benchmarks underpin robotic manipulation research. RoboTwin [9], [10], RoboCasa [11], ManiSkill3 [12], AutoBio [13], UniVTAC [14], DexGarmentLab [15], BEHAVIOR-1K [16], and SIMPLER [17] offer diverse tasks, yet most are short-horizon and solvable without historical observations. Among memory-related benchmarks, MemoryBench [6] contains few memory-dependent tasks and suffers from poor reproducibility and unclear design principles; MIKASA [7] offers a larger collection but is tailored to reinforcement learning; LIBERO-Long [8] and RoboCerebra [18] are long-horizon yet keep task-relevant information observable throughout, so they do not explicitly require memory. In contrast, RMBench explicitly stratifies memory requirements, enabling systematic analysis of memory-based policies across different difficulty levels.

B. Robotic Manipulation Policies

Generative models and imitation learning have produced a wide range of manipulation policies with strong single-task performance [19], [20], [21], [22], [23], [24], [25]. Many adopt Vision-Language-Action (VLA) formulations [26], [27], [28], [29], [30], [31], pretraining on large-scale robot data for improved generalization, with representatives including Pi0.5 [32], Pi0.6 [1], RDT2 [2], and X-VLA [33]; complementary efforts decouple high-level cognition from action synthesis [34] or learn latent action world models that predict future dynamics [35]. However, most existing policies rely on fixed-length observation histories and cannot selectively retain task-relevant information over long horizons. This motivates memory-aware approaches such as MemoryVLA [3], MemER [4], CronusVLA [5], and SAM2Act [6], which aggregate, select, or consolidate past observations, while RoboMemory [36] organizes spatial, temporal, episodic, and semantic memory into a parallel multi-module architecture that conditions high-level planning rather than the low-level action decoder. Building on this line of work, we propose Mem-0, a modular memory-enabled policy for systematic ablation of memory components and architectural choices.

III. RMBench

In this section, we introduce **Task Memory Complexity**, a principled criterion for characterizing memory requirements in robotic manipulation tasks and guiding benchmark design. Based on this, we propose **RMBench**, a simulation benchmark comprising nine manipulation tasks with varying memory demands, designed to support controlled evaluation across different levels of task memory complexity.

A. Task Memory Complexity (TMC)

Robotic manipulation frequently operates under partial observability: occlusions, delayed effects, and state aliasing make the current observation alone insufficient to determine task progress or the correct next action, and the information required to resolve this ambiguity generally lies not in a contiguous window of recent frames but in a few observations from arbitrary earlier time steps. Existing benchmarks assess memory only indirectly, through the measured performance of particular policy architectures, and therefore provide no task-centric criterion for how much past information a task actually requires. We address this gap with **Task Memory Complexity (TMC)**, which quantifies the minimum number of past observations that an optimal policy must retain.

a) Setup: We model a manipulation task as a partially observable Markov decision process (POMDP) with latent state s_t, action a_t, and observation o_t. For the tasks considered here, the reward is the sparse success indicator specified by the task goal g, so that optimality (maximal expected discounted return) is well defined. The interaction history up to time t is $h_t = (o_{1:t},\, a_{1:t-1})$.

b) Bounded memory: Rather than characterizing memory through a particular learned representation, we characterize it through how many past observations a policy is permitted to access. A *memory read of capacity* k is a map

$$\mu_k(h_t) = \big(o_{\tau_1(h_t)},\, \ldots,\, o_{\tau_k(h_t)}\big), \quad \tau_j(h_t) < t, \qquad (1)$$

where each selection rule τ_j may depend arbitrarily on h_t. A policy may thus choose *which* k past observations to retain, but not how many. Let $\mathcal{F}_k$ denote the set of all such maps, with $\mathcal{F}_0$ containing only the empty read. Because the selection rules are unconstrained, which observations matter is an outcome of the minimization below rather than an input to it.

c) Definition: The **TMC** of a task is

$$\begin{aligned}\mathrm{TMC} = \min\big\{\, k \geq 0 \;\big|\; \exists\, \mu_k \in \mathcal{F}_k,\ \exists\, \pi \text{ s.t.} \\ \pi\big(a_t \mid o_t, \mu_k(h_t), g\big) \text{ is optimal} \,\big\}.\end{aligned} \qquad (2)$$

A task therefore satisfies $\mathrm{TMC} = k$ if and only if optimal behavior is attainable while retaining k past observations but not fewer. Note that the policy in (2) conditions only on the current observation, the memory read, and the goal; it has access neither to the latent state s_t nor to any privileged phase label.

d) Task annotation: We annotate tasks as $M(0)$, $M(1)$, or $M(n)$ with $n \geq 2$, according to their TMC. $M(0)$ tasks are memory-free, in the sense that the current observation determines the optimal action; this is the regime targeted

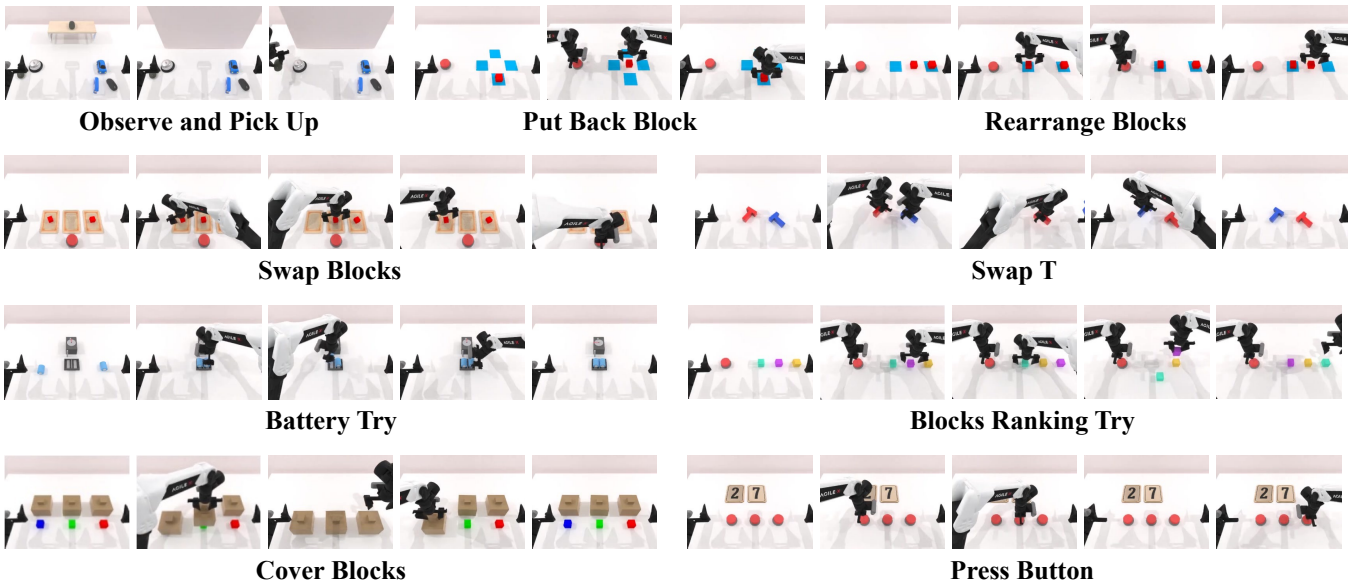


Fig. 1. **RMBench Tasks.** We illustrate the nine memory-dependent tasks in RMBench together with their key execution steps. The $M(1)$ tasks include *Observe and Pick Up*, *Rearrange Blocks*, *Put Back Block*, *Swap Blocks*, and *Swap T*; these tasks require retaining a single key observation to resolve state ambiguity across different execution stages. The $M(n)$ tasks include *Battery Try*, *Blocks Ranking Try*, *Cover Blocks*, and *Press Button*; these tasks involve repeated exploration, trial-and-error interaction, or task-specific event counting, and therefore require memory retention and retrieval over extended horizons.

by most existing manipulation benchmarks, including the standard RoboTwin 2.0 task suite. $M(1)$ tasks require a single past observation to disambiguate the current state. $M(n)$ tasks require two or more non-local observations or event records, and consequently exhibit multi-step temporal dependencies. RMBench deliberately populates the two memory-dependent levels, $M(1)$ and $M(n)$.

e) Practical estimation of TMC: Although observations are continuous in time, TMC is a task-level quantity rather than a discretization of physical time. The selection rules τ_j index semantic events, such as an initially observed configuration, a previous placement attempt, a failed insertion trial, or the number of completed button presses, so n counts distinct semantic memories rather than raw frames. In practice we estimate TMC by enumerating the decision points at which the current observation admits more than one admissible action, identifying for each such point the past observations that resolve the ambiguity, and then discarding redundant items. The number of remaining independent items is the TMC of the task.

f) Invariance to task decomposition: Any long-horizon task can be made to appear memory-free by decomposing it into primitives and supplying oracle phase labels. Definition (2) excludes this degenerate case by construction, because the policy is not permitted to condition on phase labels. Equivalently, TMC is annotated at the coarsest level at which the task goal g is expressed as a single instruction. For example, *Put Back Block* requires the initial pad of the block to be recalled after the block has been moved to the center and a button has been pressed: the trajectory can be split into visually guided primitives, yet the terminal placement still depends on one earlier observation, so the task is annotated as $M(1)$. *Blocks Ranking Try*, in contrast, requires the outcomes of several previous attempts to be accumulated, and is therefore annotated as $M(n)$.

B. RMBench System Design

Based on TMC, we design nine memory-dependent manipulation tasks, including five $M(1)$ tasks and four $M(n)$ tasks, with representative key frames shown in Fig. 1. RMBench is developed within the RoboTwin 2.0 [9] framework and built on the SAPIEN [37] simulation engine, which supports automated data synthesis and integrated policy evaluation in a unified pipeline. This design enables scalable demonstration generation as well as consistent and reproducible benchmarking of manipulation policies. In addition, RMBench provides fine-grained language annotations aligned with each action–observation pair. These annotations assign explicit linguistic descriptions to low-level interactions and state transitions, and supply the subtask-level supervision used by the hierarchical policies evaluated in Sec. V-A. Because this supervision is a property of the benchmark rather than of any single method, it is available to all policies under evaluation.

IV. Mem-0 Policy

In this section, we present **Mem-0**, a modular memory-oriented robotic policy designed for systematic analysis of memory mechanisms. As shown in Fig. 2, Mem-0 consists of a **Planning Module** (Sec. IV-A) that performs subtask-level reasoning over a key memory window, and an **Execution Module** (Sec. IV-B) that executes the current subtask using an anchor memory and a sliding memory window. The two modules are connected by a **Subtask End Classifier** (Sec. IV-C), which enables closed-loop planning and execution.

A. Planning Module

In long-horizon manipulation, end-to-end VLA models are prone to error accumulation and trajectory drift, which prior

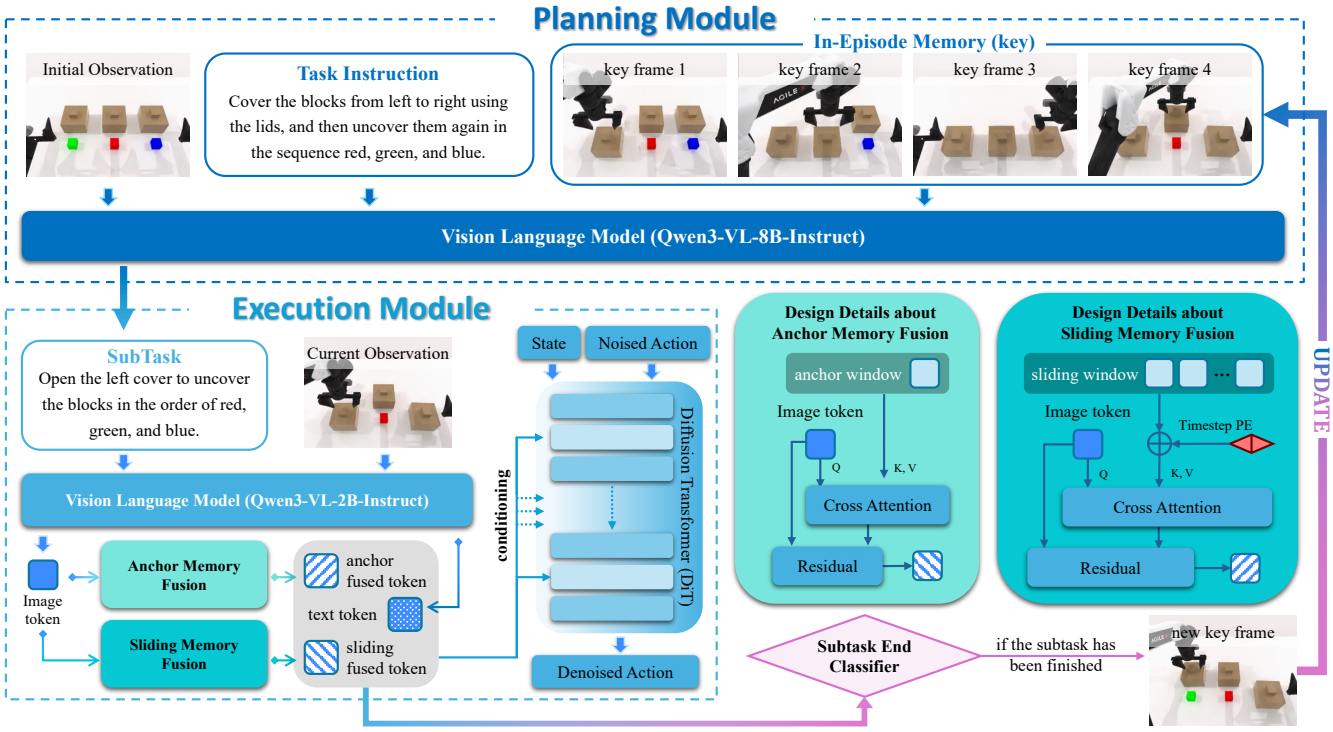


Fig. 2. **Mem-0 Pipeline.** Mem-0 comprises a Planning Module and an Execution Module linked by a Subtask End Classifier. The Planning Module generates high-level subtasks from the task instruction, the initial observation, and the key memory window (In-Episode Memory), while the Execution Module produces low-level actions from the current observation, subtask, and fused anchor and sliding memories through a flow-matching policy. Upon subtask completion, a key frame is appended to the key memory window, enabling iterative planning and execution until the task terminates.

work [4], [30] mitigates through subtask decomposition. Such approaches remain insufficient in memory-dependent settings such as the $M(n)$-type tasks in RMBench, where inferring the next subtask requires reasoning over multiple previously completed ones, for example by tracking the number of button presses. We therefore introduce a *key memory window* $\mathcal{M}_{\mathrm{K}}$ that aggregates completed subtasks and enables memory-aware subtask reasoning.

Concretely, a vision-language model $\mathcal{V}_{\text{plan}}$ predicts the i-th subtask as $s_i = \mathcal{V}_{\text{plan}}\big(o_0,\, g,\, \mathcal{M}_{\mathrm{K}}^{i-1}\big)$, where o_0 is the episode-initial RGB observation, g is the natural-language task goal, and $\mathcal{M}_{\mathrm{K}}^{i-1} = \{(s_j,\, o_j^{\text{end}})\}_{j=1}^{i-1}$ collects the textual description s_j and the terminal observation o_j^{end} of every completed subtask. Conditioning on $\mathcal{M}_{\mathrm{K}}^{i-1}$ lets the model reason explicitly over previously executed subtasks and their visual outcomes, beyond single-frame planning.

Unlike approaches that infer a new subtask at every frame and thus require $O(T)$ planning calls over a T-timestep horizon, the Planning Module is invoked only upon subtask termination, as detected by the Subtask End Classifier (Sec. IV-C); for a task with $N \ll T$ subtasks this requires only $O(N)$ invocations, so the Execution Module runs at its native control rate without being throttled by planning latency.

B. Execution Module

Although subtask-level planning is effective for many memory-dependent tasks, some tasks do not admit a useful subtask decomposition: in $M(1)$-type tasks such as *Swap T*, the target placement orientation of the T-shaped object cannot be specified reliably in language, and overly fine-grained decomposition increases annotation effort and planning latency. To address these limitations, we incorporate an **anchor memory** $\mathcal{M}_{\mathrm{A}}$ and a **sliding memory window** $\mathcal{M}_{\mathrm{S}}$ within the Execution Module, which together allow $M(1)$-type tasks to be handled without any subtask decomposition by maintaining one persistent memory slot alongside a short-term buffer.

The Execution Module executes the current subtask with a flow-matching policy conditioned on multimodal perception and memory. A vision-language model $\mathcal{V}_{\text{exec}}$ encodes the current observation o_t and the active subtask instruction s_i into image and text token embeddings, and mean pooling yields the compact latents $\mathbf{z}_t^{\text{img}}$ and $\mathbf{z}_t^{\text{text}}$. The image latent is then read against the two memory buffers with a shared residual cross-attention operator, using $\mathbf{z}_t^{\text{img}}$ as the query and the buffer contents as keys and values,

$$\tilde{\mathbf{z}}_t^{\,l} = \mathrm{CrossAttn}\Big(\mathbf{z}_t^{\text{img}},\, \mathcal{B}_t^{\,l}\Big) + \mathbf{z}_t^{\text{img}}, \tag{3}$$

for $l \in \{\text{anchor}, \text{slide}\}$, where $\mathcal{B}_t^{\text{anchor}} = \mathcal{M}_{\mathrm{A}}$ and $\mathcal{B}_t^{\text{slide}} = \mathcal{M}_{\mathrm{S}}^{t}$. For the sliding read, a learned timestep positional encoding is added to the keys and values so that the policy can distinguish recent from older entries; the anchor read needs none, since its content is by construction the earliest latent of the subtask. The two fused tokens are concatenated with the text token into the conditioning vector $\mathbf{c}_t = [\,\tilde{\mathbf{z}}_t^{\text{anchor}};\, \tilde{\mathbf{z}}_t^{\text{slide}};\, \mathbf{z}_t^{\text{text}}\,]$.

The two buffers are maintained differently. The sliding window keeps the K most recent image latents, $\mathcal{M}_{\mathrm{S}}^{t+1} =$

$\mathrm{Trunc}_K\big((\mathcal{M}_{\mathrm{S}}^t,\, \mathbf{z}_t^{\mathrm{img}})\big)$, where $\mathrm{Trunc}_K(\cdot)$ discards all but the K most recent elements. The anchor memory holds a single slot, $\mathcal{M}_{\mathrm{A}} = \{\mathbf{z}_{t_i}^{\mathrm{img}}\}$, fixed at the image latent of the first control timestep t_i of subtask i and left unchanged while that subtask executes. When a subtask transition is declared (Sec. IV-C), both buffers are cleared, $\mathcal{M}_{\mathrm{A}} \leftarrow \varnothing$ and $\mathcal{M}_{\mathrm{S}} \leftarrow \varnothing$, so that memory persisting across subtasks is carried exclusively by the key memory window.

The two reads are not symmetric, which bears directly on the ablations in Sec. V-B. Because $|\mathcal{M}_{\mathrm{A}}| = 1$, the attention weights of the anchor read are identically one, and (3) reduces to a learned projection of the subtask-initial latent added to the current latent: the anchor read performs no selection, and its benefit stems from the persistence of that context rather than from retrieval. The sliding read, in contrast, attends over K entries and performs genuine temporal selection. Enlarging the anchor buffer to support selective retrieval is left to future work.

For action generation we employ a diffusion transformer (DiT) trained with a flow-matching objective, with action horizon $H = 30$ over the $d_a = 16$-dimensional bimanual action space. Let $\mathbf{a} \in \mathbb{R}^{H \times d_a}$ denote the ground-truth action chunk $\mathbf{a}_{t:t+H-1}$. During training we draw $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and a flow time τ, interpolate linearly between noise and data, and predict the velocity of the path from the proprioceptive state $\mathbf{x}_t$ and the memory-conditioned vector $\mathbf{c}_t$,

$$\mathbf{a}^\tau = (1-\tau)\,\epsilon + \tau\,\mathbf{a}, \quad \hat{\mathbf{v}} = \mathrm{DiT}(\mathbf{a}^\tau,\, \tau,\, \mathbf{x}_t,\, \mathbf{c}_t)\,, \quad (4)$$

with regression target $\mathbf{v} = \mathbf{a} - \epsilon$. Flow times follow a shifted Beta distribution, $\tau = (s-u)/s$ with $u \sim \mathrm{Beta}(1.5, 1.0)$ and $s = 0.999$, which concentrates supervision near the data end of the path. At deployment the chunk is produced by Euler integration of (4) from ϵ over eight steps, and multiple actions from each chunk are executed before the next inference call, sustaining 5–10 Hz control on a single NVIDIA A800 GPU.

C. Subtask End Classifier

To close the loop between the Planning and Execution Modules, we introduce a *Subtask End Classifier* that detects subtask completion. A lightweight multilayer perceptron maps the conditioning vector $\mathbf{c}_t$ to a logit $\ell_t = \mathrm{MLP}(\mathbf{c}_t)$, thresholded into the binary termination signal $\mathcal{C}_{\mathrm{end}}(\mathbf{c}_t) = \mathbf{1}[\sigma(\ell_t) \geq 0.5]$, where σ is the logistic function; because it shares $\mathbf{c}_t$ with the action head, the classifier consumes no privileged simulator information. Acting on a single positive prediction is not robust, so we accumulate the count $n_t = \sum_{j=t_i}^{t} \mathcal{C}_{\mathrm{end}}(\mathbf{c}_j)$ of termination signals raised since the current subtask began, and declare a transition once $n_t \geq L_{\mathrm{sw}}$ with $L_{\mathrm{sw}} = 8$, which suppresses isolated false positives while keeping transitions responsive. On a transition the counter is reset and both buffers are cleared (Sec. IV-B); the same reset is applied during training, except that the $L_{\mathrm{mem}} = 8$ termination frames that trigger it are read from the ground-truth annotations rather than predicted. Upon transition, the final observation $o_i^{\mathrm{end}} = o_t$ and the subtask description s_i are appended to $\mathcal{M}_{\mathrm{K}}^i$ and passed to the Planning Module for the next round of subtask-level reasoning.

D. Training Objective

The Execution Module and the Subtask End Classifier are trained jointly on expert demonstrations $\mathcal{D}$, with $y_t \in \{0, 1\}$ the ground-truth termination label derived from the language annotations of Sec. III-B:

$$\mathcal{L} = \underbrace{\mathbb{E}\big[\|\hat{\mathbf{v}} - \mathbf{v}\|_2^2\big]}_{\mathcal{L}_{\mathrm{act}}} + \lambda\, \underbrace{\mathbb{E}\big[(1-p_t)^{\gamma_{\mathrm{f}}}\, \mathrm{BCE}_w(\ell_t,\, y_t)\big]}_{\mathcal{L}_{\mathrm{end}}}. \quad (5)$$

Here $\mathcal{L}_{\mathrm{act}}$ is the flow-matching regression of (4) onto the target velocity, with expectations taken over $\mathcal{D}$ and the sampled (ϵ, τ). In $\mathcal{L}_{\mathrm{end}}$, BCE_w is binary cross-entropy on the logit ℓ_t with a positive-class weight w that compensates for the rarity of termination frames, p_t is the predicted probability of the ground-truth class, and the focal exponent γ_{f} down-weights easy negatives; the scalar λ balances the two terms. The Planning Module is fine-tuned separately with a standard autoregressive language-modeling loss on subtask descriptions.

V. Experiment

We design a set of experiments to validate three key objectives: (1) to evaluate the performance of existing manipulation policies and Mem-0 on RMBench, thereby characterizing their ability to handle memory-dependent tasks across different levels of difficulty; (2) to conduct systematic ablation studies on the Mem-0 architecture in order to analyze how different module designs affect performance on memory-intensive manipulation tasks; and (3) to perform real-world robotic experiments to assess the effectiveness and generalization of Mem-0 beyond simulation.

A. Evaluation of Policies on RMBench

We benchmark eight policies on RMBench in three groups: DP and ACT trained from scratch, the large-scale pretrained Pi0.5, X-VLA, and QwenOFT, and the memory-based MemER, MemoryVLA, and our Mem-0. QwenOFT and MemoryVLA are implemented with StarVLA [38] (MemoryVLA on the OpenVLA backbone), and all policies are trained and evaluated through the unified interface of XPolicyLab [39], which our released code follows. Since MemER was developed for a single arm, we adapt its action space to the dual-arm setting of RoboTwin 2.0 while leaving its memory mechanism unchanged. Each method is trained on the same 50 expert demonstrations per task (with the objective of Sec. IV-D for Mem-0) and evaluated over 100 rollout episodes; Table I reports success rates.

Two protocol aspects shape how the comparison should be read. First, Mem-0 and MemER are hierarchical and consume the subtask annotations and termination labels provided by RMBench, whereas the remaining policies are trained end-to-end from observation–action pairs; this supervision is a property of the benchmark and available to any policy, but the gap between Mem-0 and the end-to-end baselines reflects both the memory architecture and this additional training signal. Second, Mem-0 runs without subtask decomposition on $M(1)$ tasks, isolating the Execution Module, and with

TABLE I

RMBENCH BENCHMARK RESULTS. SUCCESS RATES ON THE NINE RMBENCH TASKS SPANNING $M(1)$ AND $M(n)$. TOTAL AVERAGE IS THE UNWEIGHTED MEAN OVER ALL NINE TASKS. (**BOLD** : BEST; UNDERLINED: SECOND-BEST; GREEN: GAIN OVER THE SECOND-BEST).

Tasks	TMC	DP	ACT	Pi0.5	X-VLA	QwenOFT	MemER	MemoryVLA	Mem-0 (ours)
Observe and Pick Up	$M(1)$	1%	1%	**9%**	**9%**	0%	7%	0%	4%
Rearrange Blocks	$M(1)$	0%	29%	13%	13%	0%	17%	22%	**89%**
Put Back Block	$M(1)$	0%	0%	11%	18%	0%	0%	50%	**90%**
Swap Blocks	$M(1)$	11%	2%	24%	16%	0%	14%	17%	**67%**
Swap T	$M(1)$	**20%**	2%	15%	3%	0%	7%	9%	14%
Average	$M(1)$	6.4%	6.8%	14.4%	11.8%	0.0%	9.0%	19.6%	**52.8% (+33.2%)**
Battery Try	$M(n)$	10%	19%	16%	26%	14%	27%	25%	**28%**
Blocks Ranking Try	$M(n)$	10%	0%	6%	1%	**37%**	0%	12%	18%
Cover Blocks	$M(n)$	0%	0%	0%	2%	0%	6%	40%	**68%**
Press Button	$M(n)$	0%	0%	0%	0%	0%	0%	0%	0%
Average	$M(n)$	5.0%	4.8%	5.5%	7.3%	12.8%	8.3%	19.3%	**28.5% (+9.2%)**
Total Average	/	5.8%	5.9%	10.4%	9.8%	5.7%	8.7%	19.4%	**42.0% (+22.6%)**

TABLE II

ABLATION STUDIES ON $M(1)$ TASKS. (**BOLD** : THE BEST RESULTS; UNDERLINED: THE SECOND-BEST RESULTS).

$M(1)$ Tasks	Mem-0 (ours)	w/o Anchor	w/o Sliding
Observe and Pick Up	**4%**	**4%**	3%
Rearrange Blocks	**89%**	73%	62%
Put Back Block	**90%**	35%	78%
Swap Blocks	**67%**	15%	39%
Swap T	14%	7%	**20%**
Average	**52.8%**	26.8%	40.4%

TABLE III

ABLATION STUDIES ON $M(n)$ TASKS. GT CLASSIFIER IS AN ORACLE AND EXCLUDED FROM THE RANKING (**BOLD** : BEST DEPLOYABLE CONFIGURATION; UNDERLINED: SECOND-BEST).

$M(n)$ Tasks	Mem-0 (ours)	w/o Key	w/o Anchor	w/o Sliding	*GT Classifier (oracle)*
Battery Try	**28%**	13%	14%	17%	*30%*
Blocks Ranking Try	**18%**	1%	0%	0%	*45%*
Cover Blocks	68%	5%	**92%**	84%	*92%*
Press Button	0%	0%	**1%**	0%	*14%*
Average	**28.5%**	4.8%	26.8%	25.3%	*45.3%*

decomposition on $M(n)$ tasks, reflecting the joint behavior of both modules; Sec. V-B separates these contributions.

Policies without explicit memory perform poorly across RMBench, confirming their difficulty on non-Markovian tasks where the current observation alone is insufficient; Fig. 3 summarizes the dominant failure mode on each task (see also the supplementary video). Averaged over tasks, Mem-0 improves success rate by 33.2 percentage points on $M(1)$ tasks and 9.2 points on $M(n)$ tasks over the strongest baseline, MemoryVLA. The asymmetry is informative: $M(1)$ tasks need only a single episode-initial observation, which the anchor memory captures directly, whereas $M(n)$ tasks depend on an accumulated event history routed through subtask-level reasoning, leaving more room for error.

Mem-0 nevertheless remains far from solving RMBench. On *Observe and Pick Up* all methods stay below 10% and pretrained policies keep a small edge: the bottleneck is fine-grained semantic discrimination, not memory. On *Swap T*, performance is limited by the precision needed to match the target orientation. On *Press Button* every policy scores zero, since a single press produces only a subtle visual change that defeats termination detection.

B. Analysis of Memory-Related Modules

We ablate Mem-0 in four ways: **w/o Anchor** and **w/o Sliding** remove the corresponding Execution-Module buffer; **w/o Key** removes the key memory window, so subtask inference conditions only on o_0 and g; and **GT Classifier** replaces the learned Subtask End Classifier with ground-truth termination signals from the simulator, reported as an oracle rather than a competing configuration since it consumes privileged information unavailable at deployment. Because Mem-0 uses no subtask decomposition on $M(1)$ tasks, only the first two apply there. Tables II and III report the $M(1)$ and $M(n)$ results.

Anchor memory. On $M(1)$ tasks the anchor is decisive: removing it lowers the average from 52.8% to 26.8%, most sharply on *Put Back Block* (90% to 35%) and *Swap Blocks* (67% to 15%). This is expected, since an $M(1)$ episode runs as a single subtask whose anchor slot holds exactly the episode-initial configuration the task requires. On $M(n)$ tasks the picture partly reverses: the average barely moves (28.5% to 26.8%), and on *Cover Blocks* removal in fact *improves* success from 68% to 92%. Under subtask decomposition the needed memory is carried by the key window, as confirmed by *Cover Blocks* collapsing to 5% under **w/o Key**; the per-subtask anchor thus adds no information and instead biases the policy toward the scene at subtask onset, which is actively

Fig. 3. **Error Analysis.** Representative failure mode and its cause on each of the nine RMBench tasks.

harmful when a subtask requires closed-loop alignment with a lid whose pose keeps changing. The conclusion is narrower than a blanket endorsement of persistent memory: anchor memory helps when it is the sole carrier of task-relevant history, and hurts when it duplicates higher-level memory.

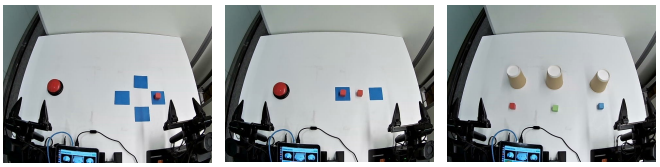
Sliding memory. Removing the sliding window reduces the $M(1)$ average from 52.8% to 40.4% and the $M(n)$ average from 28.5% to 25.3%, again not uniformly: the ablated model is better on *Swap T* (20% versus 14%), where sensitivity to the initial orientation and fairly stereotyped motion favor shifting reliance onto the anchor slot, and on *Cover Blocks* (84% versus 68%), consistent with the interference above. Together, the two ablations show that the marginal value of each buffer depends on whether the same information is already available elsewhere in the hierarchy, and that naively adding memory pathways can degrade performance.

Key memory. Removing the key window is unambiguously damaging: the $M(n)$ average falls from 28.5% to 4.8%, with *Blocks Ranking Try* dropping from 18% to 1% and *Cover Blocks* from 68% to 5%. Without completed subtasks and their visual outcomes, the Planning Module must infer the next subtask from the episode-initial observation alone, which cannot disambiguate how many attempts have been made or which blocks have been handled. Key memory thus supplies the multi-step history that $M(n)$ tasks actually require, and is the only component whose removal degrades every $M(n)$ task on which any method makes progress.

Subtask End Classifier. The classifier keeps planning at $O(N)$ calls, but its accuracy is the dominant bottleneck of the current system: with ground-truth termination signals the $M(n)$ average rises from 28.5% to 45.3% (*Blocks Ranking Try* 18% to 45%, *Press Button* 0% to 14%). Since the oracle changes nothing about the memory representation, this 16.8-point gap stems entirely from mistimed subtask transitions, and it exceeds the gap between Mem-0 and the strongest baseline on the same tasks. We therefore regard robust subtask termination detection, rather than richer memory representation, as the most consequential direction for improving hierarchical memory-dependent policies.

C. Real-World Experiments

We evaluate Mem-0 on three physical tasks aligned with RMBench, *Put Back Block*, *Rearrange Blocks*, and *Cover Blocks*, against DP, ACT, and Pi0.5, with all data collected on the X-One dual-arm platform (Fig. 4). Each policy is trained on 100 teleoperated demonstrations per task and evaluated over 40 rollout trials; Table IV reports success rates.



Put Back Block | Rearrange Blocks | Cover Blocks

Fig. 4. **Real-world Tasks Visualization.** Setups for the three physical tasks used in the real-world evaluation.

Mem-0 attains the highest success rate on all three tasks, averaging 22.5% against 5.8% for Pi0.5, while DP and ACT complete no trial. Given the low absolute values, we read this experiment as evidence that the memory mechanism transfers to hardware rather than as a demonstration

TABLE IV

REAL-WORLD EVALUATION RESULTS. SUCCESS RATES ON THE THREE PHYSICAL TASKS (**BOLD**: BEST).

Tasks	DP	ACT	Pi0.5	Mem-0 (ours)
Put Back Block	0.0%	0.0%	10.0%	**17.5%**
Rearrange Blocks	0.0%	0.0%	7.5%	**37.5%**
Cover Blocks	0.0%	0.0%	0.0%	**12.5%**
Average	0.0%	0.0%	5.8%	**22.5%**

of deployable reliability. Most Mem-0 failures stem from imprecise block manipulation rather than incorrect high-level planning, consistent with the simulation finding that memory is not the limiting factor for low-level accuracy; we attribute the low absolute performance to the higher behavioral variability of teleoperated demonstrations and to training from scratch without manipulation pretraining. Two limitations remain: the tasks cover only the $M(1)$ and $M(n)$ patterns that are straightforward to instrument physically, and no memory-based baseline is included, as neither MemER nor MemoryVLA was deployable on our platform.

VI. CONCLUSION

In this paper, we present RMBench (benchmark) and Mem-0 (policy) to systematically evaluate memory in robotic manipulation, revealing the memory limitations of existing policies and how different architectural choices (such as anchor memory, sliding memory, and key memory) affect memory performance, thereby providing preliminary insights into the principled integration of memory mechanisms for effective memory-dependent robotic manipulation. As for future work, promising directions include improved memory representation and fusion, more robust subtask termination criteria, and pretraining to enhance semantic understanding and generalization. We hope RMBench fosters principled progress toward scalable, memory-aware robotic manipulation.

REFERENCES

[1] Physical Intelligence *et al.*, "$\pi^*_{0.6}$: a VLA that learns from experience," *arXiv preprint arXiv:2511.14759*, 2025.

[2] RDT Team, "RDT2: Enabling zero-shot cross-embodiment generalization by scaling up UMI data," September 2025. [Online]. Available: https://github.com/thu-ml/RDT2

[3] H. Shi, B. Xie, Y. Liu *et al.*, "MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation," *arXiv preprint arXiv:2508.19236*, 2025.

[4] A. Sridhar, J. Pan, S. Sharma, and C. Finn, "MemER: Scaling up memory for robot control via experience retrieval," *arXiv preprint arXiv:2510.20328*, 2025.

[5] H. Li, S. Yang, Y. Chen *et al.*, "Towards efficient and robust manipulation via multi-frame vision-language-action modeling," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.

[6] H. Fang, M. Grotz, W. Pumacay *et al.*, "SAM2Act: Integrating visual foundation model with a memory architecture for robotic manipulation," *arXiv preprint arXiv:2501.18564*, 2025.

[7] E. Cherepanov, N. Kachaev, A. K. Kovalev, and A. I. Panov, "Memory, benchmark & robots: A benchmark for solving complex tasks with reinforcement learning," *arXiv preprint arXiv:2502.10550*, 2025.

[8] B. Liu, Y. Zhu, C. Gao *et al.*, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," *arXiv preprint arXiv:2306.03310*, 2023.

[9] T. Chen, Z. Chen, B. Chen *et al.*, "RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation," *arXiv preprint arXiv:2506.18088*, 2025.

[10] Y. Mu, T. Chen, Z. Chen *et al.*, "RoboTwin: Dual-arm robot benchmark with generative digital twins," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 27 649–27 660.

[11] S. Nasiriany, A. Maddukuri, L. Zhang *et al.*, "RoboCasa: Large-scale simulation of everyday tasks for generalist robots," *arXiv preprint arXiv:2406.02523*, 2024.

[12] S. Tao, F. Xiang, A. Shukla *et al.*, "ManiSkill3: GPU parallelized robotics simulation and rendering for generalizable embodied AI," *arXiv preprint arXiv:2410.00425*, 2025.

[13] Z. Lan, Y. Jiang, R. Wang *et al.*, "AutoBio: A simulation and benchmark for robotic automation in digital biology laboratory," *arXiv preprint arXiv:2505.14030*, 2025.

[14] B. Chen, W. Wan, T. Chen *et al.*, "UniVTAC: A unified simulation platform for visuo-tactile manipulation data generation, learning, and benchmarking," *arXiv preprint arXiv:2602.10093*, 2026.

[15] Y. Wang, R. Wu, Y. Chen *et al.*, "DexGarmentLab: Dexterous garment manipulation environment with generalizable policy," *arXiv preprint arXiv:2505.11032*, 2025.

[16] C. Li, R. Zhang, J. Wong *et al.*, "BEHAVIOR-1K: A human-centered, embodied AI benchmark with 1,000 everyday activities and realistic simulation," *arXiv preprint arXiv:2403.09227*, 2024.

[17] X. Li, K. Hsu, J. Gu *et al.*, "Evaluating real-world robot manipulation policies in simulation," *arXiv preprint arXiv:2405.05941*, 2024.

[18] S. Han, B. Qiu, Y. Liao *et al.*, "RoboCerebra: A large-scale benchmark for long-horizon robotic manipulation evaluation," *arXiv preprint arXiv:2506.06677*, 2025.

[19] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," *arXiv preprint arXiv:2304.13705*, 2023.

[20] C. Chi, Z. Xu, S. Feng *et al.*, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.

[21] Y. Ze, G. Zhang, K. Zhang *et al.*, "3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations," *arXiv preprint arXiv:2403.03954*, 2024.

[22] T. Chen, Y. Mu, Z. Liang *et al.*, "G3Flow: Generative 3D semantic flow for pose-aware and generalizable object manipulation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 1735–1744.

[23] G. Lu, Z. Gao, T. Chen *et al.*, "ManiCM: Real-time 3D diffusion policy via consistency model for robotic manipulation," *arXiv preprint arXiv:2406.01586*, 2024.

[24] Y. Su, X. Zhan, H. Fang *et al.*, "Dense policy: Bidirectional autoregressive learning of actions," *arXiv preprint arXiv:2503.13217*, 2025.

[25] G. Chen, Q. Liang, S. Zhu, T. Chen, Z. Xiao, Y. Xie, L. Zhang, P. Luo, R. Xu, and W. Ding, "Mopa: Coordinated mobile manipulation via subsystem-specific perception alignment," 2026. [Online]. Available: https://arxiv.org/abs/2609.12081

[26] J. Wen, Y. Zhu, J. Li *et al.*, "DexVLA: Vision-language model with plug-in diffusion expert for general robot control," *arXiv preprint arXiv:2502.05855*, 2025.

[27] M. Lin, P. Ding, S. Wang *et al.*, "HiF-VLA: Hindsight, insight and foresight through motion representation for vision-language-action models," *arXiv preprint arXiv:2512.09928*, 2025.

[28] Z. Liang, Y. Li, T. Yang *et al.*, "Discrete diffusion VLA: Bringing discrete diffusion to action decoding in vision-language-action policies," *arXiv preprint arXiv:2508.20072*, 2025.

[29] W. Shen, Y. Liu, Y. Wu *et al.*, "Expertise need not monopolize: Action-specialized mixture of experts for vision-language-action learning," *arXiv preprint arXiv:2510.14300*, 2025.

[30] J. Wen, Y. Zhu, M. Zhu *et al.*, "DiffusionVLA: Scaling robot foundation models via unified diffusion and autoregression," in *Forty-second International Conference on Machine Learning*, 2025.

[31] J. Wen, Y. Zhu, J. Li *et al.*, "TinyVLA: Toward fast, data-efficient vision-language-action models for robotic manipulation," *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3988–3995, 2025.

[32] Physical Intelligence *et al.*, "$\pi_{0.5}$: a vision-language-action model with open-world generalization," *arXiv preprint arXiv:2504.16054*, 2025.

[33] J. Zheng, J. Li, Z. Wang *et al.*, "X-VLA: Soft-prompted transformer

as scalable cross-embodiment vision-language-action model," *arXiv preprint arXiv:2510.10274*, 2025.
[34] Q. Li, Y. Liang, Z. Wang *et al.*, "CogACT: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation," *arXiv preprint arXiv:2411.19650*, 2024.
[35] H. Bi, H. Tan, S. Xie *et al.*, "Motus: A unified latent action world model," *arXiv preprint arXiv:2512.13030*, 2025.
[36] M. Lei, H. Cai, Z. Cui *et al.*, "RoboMemory: A brain-inspired multi-memory agentic framework for interactive environmental learning in physical embodied systems," *arXiv preprint arXiv:2508.01415*, 2025.
[37] F. Xiang, Y. Qin, K. Mo *et al.*, "SAPIEN: A simulated part-based interactive environment," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 11 097–11 107.
[38] StarVLA Community, "StarVLA: A Lego-like codebase for vision-language-action model developing," *arXiv preprint arXiv:2604.05014*, 2026.
[39] XPolicyLab Community *et al.*, "XPolicyLab: A unified standard and open ecosystem for robot policy evaluation and deployment," *arXiv preprint arXiv:2608.09892*, 2026.
[40] Y. Zheng, R. Zhang, J. Zhang *et al.*, "LlamaFactory: Unified efficient fine-tuning of 100+ language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. Bangkok, Thailand: Association for Computational Linguistics, 2024. [Online]. Available: http://arxiv.org/abs/2403.13372
[41] W. Kwon, Z. Li, S. Zhuang *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

APPENDIX I
RMBENCH TASK DESCRIPTIONS

TABLE V
TASK DESCRIPTIONS OF THE RMBENCH BENCHMARK.

Task	**Description**
Observe and Pick Up	The robot observes a reference object, remembers it after it is hidden, and picks up the matching object from the table.
Rearrange Blocks	The robot moves a middle block to a pad, presses a button, and moves another block back to the middle position.
Put Back Block	The robot moves a block to the center, presses a button, and returns the block to its original pad.
Swap Blocks	The robot swaps two blocks using an empty pad and then presses the confirmation button.
Swap T	The robot swaps the positions and orientations of two colored T-shaped blocks.
Battery Try	The robot repeatedly tries different battery insertion orders and orientations until both batteries are correctly inserted.
Blocks Ranking Try	The robot repeatedly arranges colored blocks and presses a button until the correct ordering is found.
Cover Blocks	The robot covers blocks from left to right, then uncovers them in the red–green–blue order.
Press Button	The robot presses two buttons according to the shown digits and then presses the confirmation button.

APPENDIX II
STABILITY AND TRANSFERABILITY OF TASK MEMORY COMPLEXITY

This section examines two properties of the Task Memory Complexity (TMC) protocol: whether it applies to tasks outside RMBench, and whether annotators other than the authors reproduce the intended labels. Both are necessary if TMC is to serve as a benchmark design criterion rather than as a post-hoc description of the tasks we happened to build.

a) Transferability to real-robot tasks: TMC is defined by the structure of the task, namely the minimum number of past observations that resolve its ambiguous decision points, and not by any policy architecture. It is therefore applicable wherever those decision points can be enumerated. To test this, we annotated 50 manipulation tasks drawn from the DROID and AgiBot World datasets. Annotation followed the protocol of Sec. III-A: enumerate the decision points at which the current observation admits more than one admissible action, identify the past observations that resolve each ambiguity, and discard redundant items. Every task in this set received a label within the existing space $\{M(0), M(1), M(n)\}$, and

TABLE VI

STABILITY OF TMC ANNOTATION. TEN INDEPENDENT PARTICIPANTS LABELED THE SAME TASK SET USING ONLY THE WRITTEN PROTOCOL OF SEC. III-A. AGREEMENT IS THE FRACTION OF TASKS MATCHING THE REFERENCE LABELS OF THE TASK DESIGNERS; AVERAGE, HIGHEST, AND LOWEST ARE TAKEN OVER THE TEN PARTICIPANTS.

Metric	Average	Highest	Lowest
Agreement (%)	96	100	80

none required extending the label space or introducing task-specific exceptions. We report this as evidence of applicability rather than as a survey: the sample was not drawn so as to be representative of either dataset, and we therefore do not report a distribution over TMC levels for it. The claim supported by this exercise is that the protocol terminates with a well-defined label on tasks it was not designed around.

b) Annotation stability study: We further conducted a study with ten independent participants who had not previously read the paper. Each participant received only the written protocol of Sec. III-A and the three-way label space $\{M(0), M(1), M(n)\}$, and then annotated the same task set without consulting the authors or one another. Performance is measured as the per-annotator fraction of tasks whose label matches the reference label assigned by the task designers. As shown in Table VI, agreement averaged 96% over the ten participants, ranging from 100% to 80%. For calibration, a labeler guessing uniformly at random over the three-way space would reach 33%.

c) Discussion: Two conclusions follow, both deliberately narrow. First, TMC applies to tasks it was not designed around, provided their decision points are enumerable; it does not provide a procedure for tasks whose success criterion is itself ambiguous. Second, participants working from the written protocol alone recovered the intended labels in the large majority of cases, which indicates that the protocol is transmissible in writing and does not depend on tacit knowledge held by the authors. This is the property required for others to extend RMBench with new tasks.

We state three limitations explicitly, since they bound how the numbers above should be read. First, the reported quantity is agreement with a reference standard rather than chance-corrected agreement among annotators. Fleiss' κ over the participants would be the stronger measure, since it separates genuine consensus from the agreement expected by chance and does not privilege the authors' labels as ground truth; computing it requires retaining the full per-task label matrix, which we leave to an expanded study. The 96% figure should therefore be read as a measure of how faithfully the protocol transfers to new annotators, and not as an accuracy or as evidence that the labels are independent of the authors' judgment. Second, the label space was closed and given to participants in advance, so the study measures consistency of application within that space rather than whether the space itself is the right abstraction. Third, the study reports aggregate agreement only, without a per-label breakdown. The $M(1)$ versus $M(n)$ boundary is the most delicate judgment the protocol requires, since it asks how finely a task decomposes into distinct semantic events, and a per-label breakdown would be needed to establish whether residual disagreement concentrates there.

APPENDIX III
VISUALIZATION OF TYPICAL BASELINE ERRORS

APPENDIX IV
TRAINING DETAILS

A. Planning Module

In the Planning Module, we fine-tune the vision-language model (Qwen3-VL-8B-Instruct) using LoRA via LlamaFactory [40] to enable reasoning over key memories. After fine-tuning, we deploy the model with vLLM [41] for efficient loading and inference. The key hyperparameters used for VLM fine-tuning are summarized in Table VII. Training is conducted on 8 NVIDIA A800 GPUs and the duration of training for a single task is approximately half an hour.

TABLE VII

HYPERPARAMETERS FOR FINE-TUNING THE MEM-0 PLANNING MODULE.

Configuration	**Value**
VLM Backbone $\mathcal{V}_{\text{plan}}$	Qwen3-VL-8B-Instruct
Fine-tuning Type	LoRA
LoRA Rank	8
Batch Size	16
Learning Rate	1.0×10^{-4}
Epochs	25
LR Scheduler	Cosine
Warmup Ratio	0.1
Dtype	bf16

B. Execution Module

In this section, we detail the training infrastructure, training organization strategy, and hyperparameter configurations employed for the Execution Module in Mem-0.

1) Model Configuration: The Execution Module uses Qwen3-VL-2B-Instruct as its vision-language backbone $\mathcal{V}_{\text{exec}}$, which is smaller than the Qwen3-VL-8B-Instruct model used by the Planning Module, since $\mathcal{V}_{\text{exec}}$ is queried at every control step whereas $\mathcal{V}_{\text{plan}}$ is queried only at subtask boundaries. The action head is a DiT-B transformer trained from scratch with a flow-matching objective, using an action horizon of $H = 30$ over a 16-dimensional bimanual action space. The Subtask End Classifier operates on the same conditioning vector $\mathbf{c}_t$ as the action head; since $\mathbf{c}_t$ concatenates the two fused memory tokens with the text token, each of width 2048, its dimension is 6144, which the classifier maps through two residual blocks ($6144 \to 2048 \to 512$) and a linear head to a scalar logit. Table VIII reports the full configuration, including the memory hyperparameters (sliding window length K, anchor buffer size, and the two accumulation thresholds L_{sw} and L_{mem}), the number of executed action steps Δ, and the classifier loss terms (w, γ_{f}, and λ). Two settings are worth highlighting.

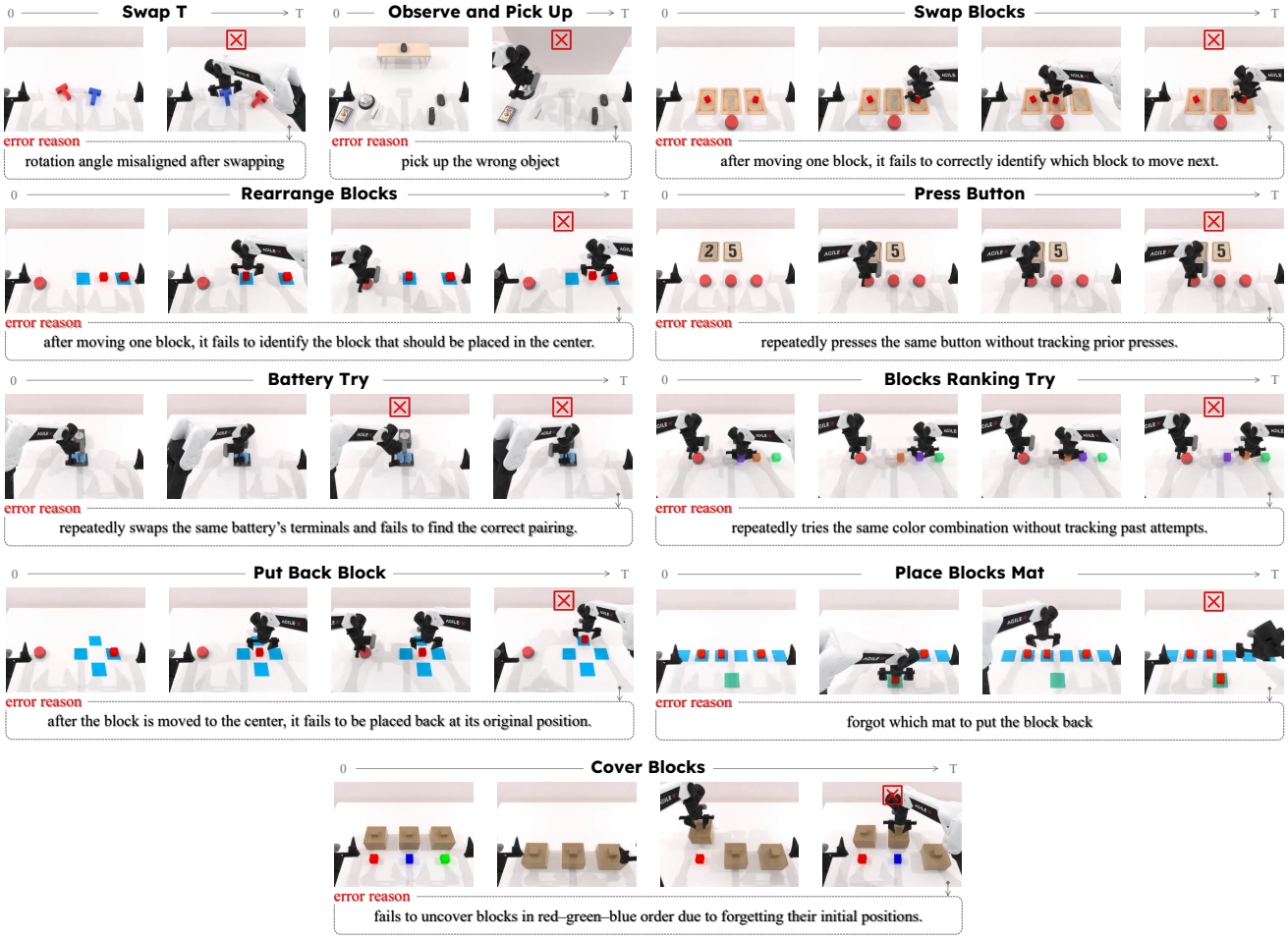


Fig. 5. **Visualization of typical baseline errors.** Because these baselines predict the next action from the current observation alone, they cannot act reliably on non-Markovian tasks in which the required information is no longer visible.

First, $\Delta = H = 30$, so each predicted chunk is consumed in full before the next inference call, which is what allows the module to sustain a 5–10 Hz control rate despite invoking a 2B vision-language backbone. Second, the classifier is trained with a positive-class weight of 10.0, since termination frames constitute a small minority of the training frames.

2) Training Infrastructure and Time Budget: The Execution Module of Mem-0 utilizes a single-task training strategy, where the model is trained from scratch for each specific task. Training is conducted on 8 NVIDIA A800 GPUs with a global batch size of 448 over 30K iterations. The duration of training for a single task is approximately 18 hours.

3) Training Organization Strategy: **Forward pass strategy.** Given the memory-centric architecture of Mem-0, we employ a specialized training methodology. Within each batch, the processes of VLM token generation and DiT-based action chunk generation are executed in parallel. Conversely, the fusion of the sliding and anchor memories requires the temporal integrity of the data, so this stage is processed serially, which keeps all frames within an episode sequentially aligned.

This arrangement keeps the VLM tokens usable for memory fusion without breaking episode ordering. We additionally maintain a global data structure during training that stores the memory state of each episode, which allows memory to be carried across batch boundaries.

Dataloader implementation. Consequently, the dataloader was custom-designed to align with this architecture. Episodes are distributed as evenly as possible across all GPUs. Each GPU processes its assigned episodes using a specific number of workers and manages the dataloader reset independently. Due to the stochastic nature of the distribution, as training iterations progress, the frames within a global batch become temporally desynchronized. This allows the model to learn simultaneously from data that span various time steps.

4) Hyperparameter Configurations: Table VIII summarizes the key training hyperparameters. To balance the learning rate requirements of different modules, we implemented a grouped learning rate strategy alongside a cosine learning rate schedule with linear warm-up. Regarding numerical precision, the VLM and Memory Bank components operate in `bfloat16`, while all other modules utilize `float32`. Furthermore, regarding visual inputs, images are resized to 224×224 and subjected to mild data augmentation via frame-independent `ColorJitter`, aimed at enhancing the model's generalization capabilities.

TABLE VIII

HYPERPARAMETERS FOR TRAINING AND DEPLOYING THE MEM-0 EXECUTION MODULE.

Configuration	Value
General Training Settings	
Batch Size (global / per GPU)	448 / 56
Iterations	30,000
Max Grad. Norm	2.5
LR Scheduler	Cosine
Warmup Ratio	0.05
Optimizer	AdamW
Momentum	$\beta_1, \beta_2 = 0.9, 0.999$
Weight Decay	0.005
Workers per GPU	2
Random Seed	42
Learning Rates	
LR (Base)	1.0×10^{-5}
LR (VLM)	1.0×10^{-5}
LR (Action Head)	1.0×10^{-4}
LR (Classifier)	1.0×10^{-4}
Min LR (Base)	1.0×10^{-6}
Min LR (VLM)	1.0×10^{-6}
Min LR (Action Head)	5.0×10^{-6}
Min LR (Classifier)	5.0×10^{-6}
Visual Input Settings	
Image Resize	224×224
Image Aug.	ColorJitter[†]

[†] ColorJitter parameters: $(0.1, 0.1, 0.1, 0)$.

Configuration	Value
Backbone and Memory	
VLM Backbone $\mathcal{V}_{\text{exec}}$	Qwen3-VL-2B-Instruct
Sliding Window Length K	30
Anchor Buffer Size	1
Memory Attention Heads	8
Memory Dropout	0.1
Transition Threshold L_{sw} (deploy)	8
Buffer Reset Threshold L_{mem} (train)	8
Action Head (Flow Matching)	
Architecture	DiT-B
Hidden Size / Layers	2048 / 16
Action Dim. d_a / State Dim.	16 / 16
Action Horizon H	30
Executed Steps Δ	30
Flow Time Dist.	$\text{Beta}(1.5, 1.0)$
Flow Time Shift s	0.999
Inference Euler Steps	8
Control Frequency	5–10 Hz
Subtask End Classifier	
MLP Widths	$6144 \to 2048 \to 512$
Dropout	0.1
Positive Weight w	10.0
Focal Exponent γ_{f}	1.0
Decision Threshold	0.5
Loss Weight λ	0.2

APPENDIX V
ADDITIONAL VISUALIZATIONS AND ANALYSIS OF FAILURE CASES IN MEM-0

Although Mem-0 improves substantially over the baselines, it remains far from solving RMBench. In this section we present representative cases in which Mem-0 fails, and identify the mechanism responsible for each, so that the residual error modes can be targeted directly by future work.

A. Failure Analysis for $M(1)$ Tasks

For $M(1)$ tasks, in addition to the failures illustrated in Fig. 5, we summarize the representative errors encountered by Mem-0 below.

Observe and Pick Up and Swap Blocks. Fig. 6 illustrates typical failure modes in the *Observe and Pick Up* task, where Mem-0 fails to accurately identify the target object. Similarly, Fig. 7 presents examples of misjudgments regarding the termination of the swapping sequence in the *Swap Blocks* task, resulting in the confirmation button being pressed at inappropriate times.

These instances indicate that the anchor memory influences the policy throughout the entire subtask horizon rather than only at its beginning, so the model must modulate how strongly that fixed context contributes to action prediction as the subtask proceeds. This is consistent with the observation in Sec. V-B that a persistent context can interfere with closed-loop visual alignment once the relevant history is already available elsewhere.

Nevertheless, the ablations confirm that anchor memory is the decisive component on $M(1)$ tasks, where its removal lowers the average success rate from 52.8% to 26.8%. The effect is most pronounced on *Put Back Block* (90% to 35%) and *Swap Blocks* (67% to 15%), the two tasks in which the episode-initial configuration is the only record of the required target.

Rearrange Blocks. Fig. 8 illustrates failure cases in the *Rearrange Blocks* task where Mem-0 performs excessive button presses, a behavior we attribute to the limitations of the sliding memory window. This is consistent with the ablations, in which removing the sliding memory reduces success on this task from 89% to 62%. Inspecting the rollouts, the frequency of redundant button presses increases markedly without sliding memory, which identifies it as a primary failure mode. The sliding memory therefore contributes substantially on this task while leaving room for refinement.

Summary. Two directions follow. First, richer fusion between the anchor and sliding memories would let the policy weight the persistent context according to how far the subtask has progressed, which is precisely what the failures above require and what a single-slot anchor read cannot express. Second, the residual errors on *Observe and Pick Up* are perceptual rather than mnemonic, so they call for stronger visual grounding in the backbone rather than

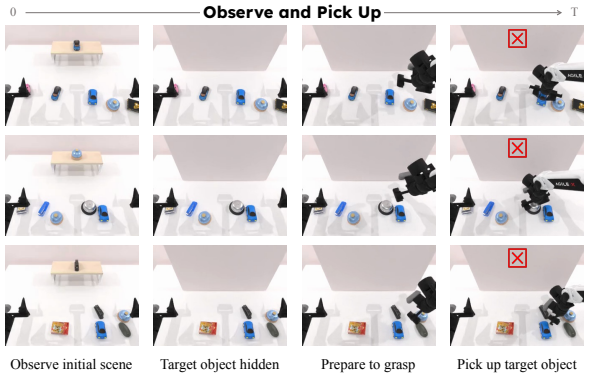


Fig. 6. **Failure examples of Observe and Pick Up. (Top)** Confused by objects with similar colors and shapes. **(Middle)** Confused by identical object morphologies. **(Bottom)** General failure to identify the target, causing the gripper to reach a position averaged over the candidate objects rather than any one of them.

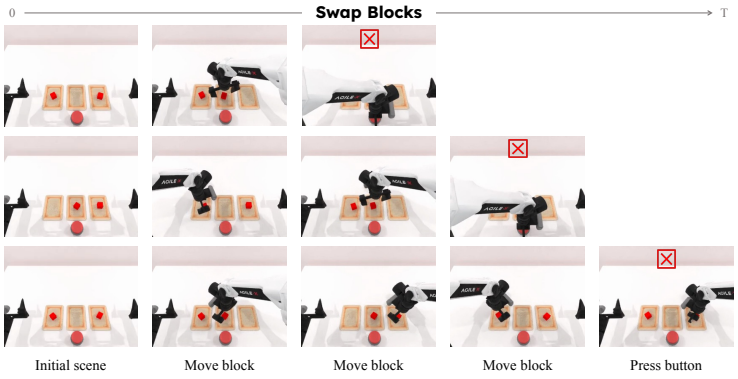


Fig. 7. **Failure examples of Swap Blocks. (Top)** Premature termination after a single subtask. **(Middle)** Premature termination after two subtasks. **(Bottom)** Failure to terminate on time, resulting in the initiation of a redundant subtask.

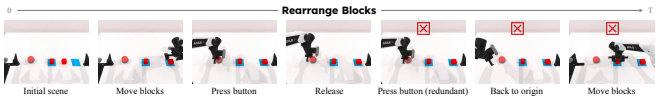


Fig. 8. **Failure examples of Rearrange Blocks.** Mem-0 redundantly presses the button, resulting in task failure.

additional memory.

B. Failure Analysis for $M(n)$ Tasks

On $M(n)$ tasks the dominant failure mechanism is the accuracy and robustness of the Subtask End Classifier, consistent with the oracle comparison of Sec. V-B, in which ground-truth termination signals raise the $M(n)$ average from 28.5% to 45.3%.

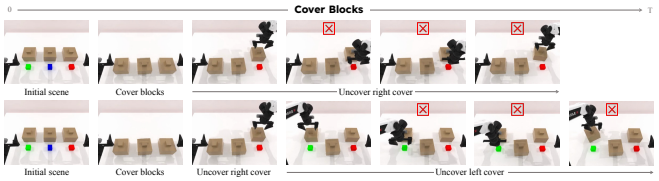


Fig. 9. **Failure examples of Cover Blocks.** The classifier fails to detect the completion of an *Uncover* ⟨*color*⟩ subtask, so no subtask transition is triggered. Because the instruction remains unchanged, the policy continues to operate under a stale task context, leading to unintended and erratic behavior.

Cover Blocks. The classifier occasionally fails to resolve the current task progress, and in particular cannot discriminate whether a given state marks the beginning or the end of a subtask, since the two are visually similar for this task. Fig. 9 shows an instance during *Cover Blocks*.

Blocks Ranking Try. A second challenge concerns button-pressing operations. We observe that introducing button presses interferes with composite tasks that are not exclusively about pressing. In *Blocks Ranking Try*, for instance, the transition between button-pressing and block-swapping is occasionally not carried out smoothly, as illustrated in Fig. 10.

The task is also unforgiving: its success criterion requires the full ranking sequence to be correct, so a single execution error causes the episode to fail. This is consistent with the low absolute scores every method obtains on this task, and with the fact that supplying oracle termination signals raises Mem-0 from 18% to 45%, the largest such improvement among the $M(n)$ tasks.

Fig. 10. **Failure examples of Blocks Ranking Try.** Upon pressing the button, the system is expected to transition to the next subtask to execute the swapping of designated blocks. However, the classifier fails to trigger this transition promptly, causing the task to stall in the `Press button` state. This leads to a coordination conflict between the dual arms: the right hand attempts to initiate manipulation while the left hand remains tethered to the button-pressing instruction.

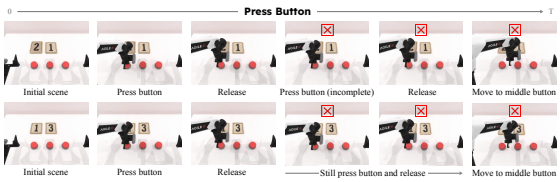


Fig. 11. **Failure examples of Press Button. (Top)** Insufficient presses: the classifier issues a false positive termination signal even though the press was unsuccessful. **(Bottom)** Excessive presses: the classifier fails to recognize a successful completion, leading to redundant execution of the same subtask.

Press Button. The *Press Button* task isolates this operation and therefore exposes its difficulty most clearly, which is why every policy in Table I scores zero on it. The button state, pressed versus unpressed, is reflected in the visual input only through very subtle cues, so the visual tokens produced by the VLM backbone do not resolve it. This places a bottleneck upstream of the classifier that no amount of memory can compensate for. Fig. 11 shows representative cases.

Battery Try. For *Battery Try* the representative failure mode is limited manipulation precision rather than memory. It comprises insertion errors when placing the battery into the slot, and difficulty selecting a grasp strategy because the

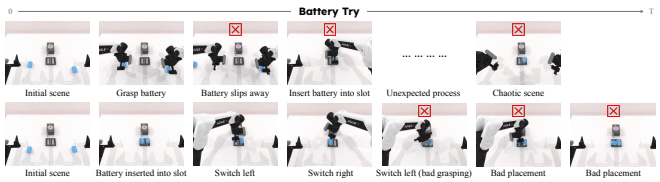


Fig. 12. **Failure examples of Battery Try. (Top)** For the horizontally oriented cylindrical battery, a suboptimal grasp pose prevents a successful lift and causes significant displacement, leading the model into unforeseen observational states. **(Bottom)** The model fails to commit to a single manipulation strategy during battery adjustment, producing an action averaged over the competing strategies and hence an improper placement in the slot.

slot itself presents very subtle visual cues, as illustrated in Fig. 12.

Summary. Two directions follow from these observations. First, since several of the failures above trace to visual cues that are too subtle to survive tokenization, proprioceptive or tactile feedback would supply the classifier with information that vision does not carry. Second, because the classifier sits at the end of the pipeline and consumes $\mathbf{c}_t$ directly, improving upstream token extraction and the fusion of the anchor and sliding memories would improve termination detection without altering the classifier itself. Both directions target the bottleneck identified in Sec. V-B rather than the memory representation.

The current design nonetheless yields clear gains on $M(n)$ tasks, most visibly on *Cover Blocks*, where Mem-0 reaches 68% against 40% for the strongest baseline.